

Parallel and Distributed Computing

Assignment 8

Instructor: Dr. Saif Nalband

Course Code: UCS645

Problem 1: GPU Architecture and Profiling

Part A: CPU vs GPU Speedup Benchmark

Vector addition was implemented on both CPU and GPU. Execution time was measured for input sizes:

- $N=210, 214, 218, 222, 226$ $N = 2^{\{10\}}, 2^{\{14\}}, 2^{\{18\}}, 2^{\{22\}}, 2^{\{26\}}$

Metrics recorded:

- CPU execution time
- GPU kernel time
- Host-to-device transfer time

Observation:

- CPU performs better for very small inputs due to lower overhead
 - GPU outperforms CPU for larger inputs
 - Crossover point occurs between 210×2^{10} and 214×2^{14}
 - Larger inputs better utilize GPU parallelism
-

Part B: Launch Configuration Analysis

Thread configurations tested:

- 64, 128, 256, 512, 1024 threads/block

Block calculation:

$$\text{blocks} = \frac{N + \text{threads} - 1}{\text{threads}}$$

$$\text{blocks} = \frac{N + \text{threads} - 1}{\text{threads}}$$

Observation:

- Best performance achieved between 128–512 threads
- Very small/large blocks reduce efficiency

Reason:

CUDA executes threads in warps of 32 → multiples of 32 give optimal utilization

Conclusion

GPU provides significant acceleration for large-scale workloads. Performance depends heavily on problem size and kernel configuration.

Problem 2: Memory Hierarchy and Optimization

Parallel Reduction Methods

- Naive reduction
- Shared memory tree reduction
- Warp shuffle reduction

Results:

- Naive approach is slow (no parallelism)
 - Shared memory improves performance significantly
 - Warp-level operations further reduce overhead
-

Bank Conflict Analysis

- Different memory strides tested
 - Conflicts occur when threads access same memory bank
 - Leads to serialized execution and slower performance
-

Conclusion

Efficient memory access and shared memory usage are critical for high GPU performance.

Problem 3: Machine Learning Kernels

Implemented Operations

- Sigmoid
- Tanh
- Leaky ReLU
- ReLU backward
- Binary Cross-Entropy loss

Verification:

- GPU outputs matched CPU results
 - All kernels executed successfully
-

Observation

- Each thread processes one element
 - Enables full parallel utilization
-

Conclusion

CUDA efficiently accelerates ML operations through parallel execution.

Problem 4: Matrix Multiplication and CNN Layers

Tiled Matrix Multiplication

- Shared memory used
 - Tile size: 16×16
 - Reduces global memory access
-

CNN Operations Implemented

- Max Pooling (2×2)
 - Batch Normalization (inference)
-

Observation

- Kernels executed successfully
 - Proper grid/block configuration ensured parallelism
-

Conclusion

GPU acceleration significantly improves performance for matrix and CNN computations.

Problem 5: CNN Training Pipeline (Simulation)

Implementation

- CUDA-based training loop
 - Dummy forward kernel simulates computation
 - Device memory used for processing
-

Training Behavior

- Loss decreases over epochs
- Accuracy increases progressively
- Demonstrates learning trend

Observation

- GPU utilized in each iteration
- Synchronization ensures correct execution

Conclusion

The program successfully simulates a CNN training pipeline and demonstrates how GPU parallelism accelerates deep learning workflows.